

Sistema de recomendación de contenidos

Base de datos para inteligencia artificial

Carrera de especialización en inteligencia artificial

Laboratorio de sistemas embebidos - Facultad de ingeniería -
Universidad de Buenos Aires

Grupo 1

Berdaguer, Carlos Agustín - a2605

Campi, Lucas - a2609

Souto, Matías - a2638

Índice

[Índice](#)

[Descripción del caso de uso y relevamiento de los datos necesarios](#)

[Consigna](#)

[Propuesta](#)

[Aplicación de prueba](#)

[Entidades principales identificadas](#)

[Clasificación de los datos](#)

[Modelo conceptual y modelo de implementación](#)

[Modelo lógico](#)

[Modelo físico](#)

[Decisiones de normalización en las relaciones](#)

[Catálogos, muchos-a-muchos, y claves subrogadas](#)

[Reference over embedding y herencia de atributos](#)

[Eventos como filas](#)

[Desnormalizaciones deliberadas](#)

[Selección y justificación de la tecnología utilizada](#)

[Implementación mínima](#)

[Datos de ejemplo y consultas representativas](#)

[Arquitectura general de datos](#)

[Capas](#)

[Seguridad, permisos y aislamiento](#)

[Escalabilidad y rendimiento](#)

[Conclusiones](#)

[A confirmar si se responde en el informe](#)

Descripción del caso de uso y relevamiento de los datos necesarios

Consigna

Una plataforma digital desea sugerir contenidos a sus usuarios, como noticias, artículos, videos, publicaciones, cursos o documentos. Las recomendaciones pueden basarse en historial de visualizaciones, búsquedas, preferencias declaradas, interacciones previas o similitud entre contenidos.

Se desea diseñar una solución de datos que permita administrar contenidos, registrar interacciones de usuarios, clasificar contenidos por categorías o etiquetas y generar recomendaciones personalizadas o contextuales.

El sistema deberá contemplar usuarios finales, editores, administradores, moderadores y analistas. También deberán registrarse publicaciones, ediciones, moderaciones, visualizaciones, recomendaciones mostradas y acciones realizadas sobre cada contenido.

Algunos datos posibles: usuarios, contenidos, categorías, visualizaciones, búsquedas, interacciones, preferencias, recomendaciones, fuentes de contenido, etiquetas, publicaciones y acciones de moderación.

Propuesta

Se propone un portal general de contenidos donde los usuarios pueden consultar materiales de distintos tipos y los creadores pueden publicarlos. El problema es encontrar información relevante dentro de un volumen y variedad crecientes. La solución combina clasificación, registro de interacciones, recomendaciones básicas y búsqueda semántica.

Aplicación de prueba

Se confeccionó una aplicación web para probar algunas de las características de los datos directamente con las bases de datos implementadas incluyendo datos de prueba y permitiendo dar de alta nuevos datos.

Inicio	Categorías	Tags	Usuarios	Contenido	Búsqueda
Contenido + Nuevo contenido					
Vista	Título	Tipo	Categoría	Creador	Acciones
	Infraestructura y Despliegue en la Nube	photo	DevOps & Cloud	diana_prince	Editar Eliminar
—	Especificación del TP Integrador BDIA	document	Bases de Datos	admin_user	Editar Eliminar
—	Curso Completo de Data Engineering & IA	course	Inteligencia Artificial	charlie_brown	Editar Eliminar
—	Novedades sobre la cursada de BDIA 2026	post	Desarrollo Web	charlie_brown	Editar Eliminar
—	Introducción a Modelos de Lenguaje LLM	article	Inteligencia Artificial	alice_smith	Editar Eliminar
	Tutorial de PostgreSQL 16 desde Cero	video	Bases de Datos	alice_smith	Editar Eliminar

Listado de contenido en aplicación de prueba.

[Inicio](#)
[Categorías](#)
[Tags](#)
[Usuarios](#)
[Contenido](#)
[Búsqueda](#)

Búsqueda semántica

usuarios quieren contenido sugerido

-- Usuario predeterminado (10) --

Buscar

Resultados para "usuarios quieren contenido sugerido"

Máx. resultados configurados: 10

Vista	Título	Tipo	Fuente	Fragmento	Similitud
—	trabajo	post	body	Una plataforma digital desea sugerir contenidos a sus usuarios, como noticias, artículos, videos, publicaciones, cursos o documentos. Las recomendaciones pueden basarse en historial de visualizaciones, búsquedas, preferencias declaradas, interacciones previas o similitud entre contenidos.	0.498
—	trabajo	post	body	interacciones previas o similitud entre contenidos. Se desea diseñar una solución de datos que permita administrar contenidos, registrar interacciones de usuarios, clasificar contenidos por categorías o etiquetas y generar recomendaciones personalizadas o contextuales. El sistema deberá contemplar usuarios finales, editores, administradores, moderadores y analistas.	0.439

Búsqueda semántica en aplicación de prueba.

Modelo físico

El modelo lógico se encuentra en [tp_integrador/docs/der.mmd](#) y muestra las tablas y relaciones del esquema. Las columnas, tipos, claves primarias, claves foráneas y restricciones se definen en [tp_integrador/sql/ddl.sql](#) como modelo físico.

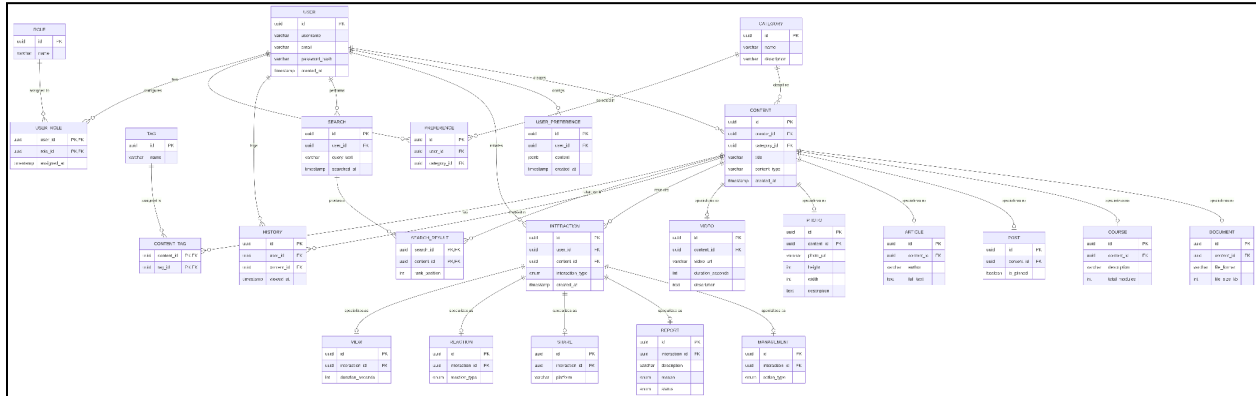


Diagrama de entidad-referencia (puede encontrarse en formato PNG en [tp_integrador/docs/der.png](#))

Decisiones de normalización en las relaciones

Catálogos, muchos-a-muchos, y claves subrogadas

Una de las primeras decisiones que hubo que tomar fue cómo representar a los usuarios, roles y permisos. Para darle prioridad al desarrollo relativo al manejo de contenido, omitimos modelar los permisos y aceptar una estructura *Role Based Access Control (RBAC)*. En ella, cada usuario tiene asignado uno o más roles que representan un conjunto de capacidades o privilegios¹. En esta entrega, damos por hecho que la aplicación va a gestionar la creación y autenticación pero nosotros no la implementamos. Dedicaremos unas palabras a la seguridad y el aislamiento más adelante.

Nuestro modelo relacional se organiza hasta 3FN. Se separan catálogos, entidades y relaciones para evitar redundancias y anomalías. **CONTENT_TAG**, **USER_ROLE** y **RESULTADO_BUSQUEDA** resuelven relaciones muchos-a-muchos. Las tablas de tipos específicos evitan columnas nulas. **creator_id** identifica al usuario creador de cada contenido y **MANAGEMENT** registra acciones posteriores de gestión. **CONTENT_EMBEDDING** es una representación derivada y controlada para optimizar la búsqueda semántica.

Otro punto a destacar es el uso de claves subrogadas vía **id UUID PRIMARY KEY DEFAULT gen_random_uuid()** en todas las tablas. Esto permite desacoplar la identidad de los atributos de negocio. Por ejemplo, si cambia el **name** de un **tag**, las claves foráneas que lo referencian no se rompen. Además evita que claves primarias compuestas se propaguen en cascada por el esquema.

¹ <https://www.ibm.com/mx-es/think/topics/rbac>

Por otro lado, entidades representadas con modelos de datos no relacionales se guardan como referencias, como los archivos binarios de fotos o videos guardados en una base de datos MinIO donde se guarda la URI de referencia, o las preferencias de la aplicación del usuario (`USER_PREFERENCE`) almacenadas como JSONB; esto último permite adaptar la aplicación con nuevas funcionalidades que dependan de estas preferencias sin necesidad de una migración masiva de datos.

Reference over embedding y herencia de atributos

Como norma general, seguimos el principio de *siempre referenciar, nunca embeber*. Hacer que las relaciones entre diferentes entidades mediadas por `CONTENT`, que es el tipo base para cualquier acción de posteo, nos permite no repetir contenido ni tener que hacer operaciones con valores posiblemente muy grandes. Tomemos como guía un caso de creación de un `POST`. Primero debe crearse la entrada en `CONTENT`:

```
INSERT INTO CONTENT (title, content_type, category_id, creator_id)
VALUES (%s, %s, %s, %s)
RETURNING id
(
    form["title"],
    form["content_type"],
    form.get("category_id") or None,
    form.get("creator_id") or None,
),
```

Donde `form` proviene de la app y contiene la información de qué se crea, quién lo crea, y cómo se llama. Una vez que se hace esa operación y se retorna su `id`, se procede a hacer el upsert del `POST`:

```
INSERT INTO POST (content_id, is_pinned, body)
VALUES (%s, %s, %s)
ON CONFLICT (content_id) DO UPDATE
SET is_pinned = EXCLUDED.is_pinned, body = EXCLUDED.body
(content_id, form.get("is_pinned") == "on", form["body"]),
```

Toda la lógica de creación de contenidos está en `tp_integrador/app/routers/content.py`

En caso de que el `content_id` ya existiera, lo va a actualizar. Los valores del `body` viven en el post pero el contenido se referencia vía el id del contenido. Este esquema desacopla la realización del contenido con su registro. Es decir, no necesitamos buscar el cuerpo del post para encontrar el post.

Otro punto fuerte de este diseño es que tenemos muchos tipos de contenido (`POST`, `HISTORY`, `SEARCH_RESULT`, `PHOTO`, `VIDEO`, etc.) que comparten atributos comunes y declaran específicos. Al abstraer en `CONTENT` el título, categoría y creador, se puede relacionar directamente al tipo de contenido via `content_type` y evitar tener una sola tabla con muchos atributos que deberían ser nullable por restricción pero en realidad, por definición, deberían ser not null. Otra forma de verlo sería que ahorramos generar una matriz dispersa en la que al crear un `POST` estaríamos obligados a dejar null atributos como `video_url`, `height`, `viewed_at`, etc. que son propios de otros tipos de contenido.

Seguimos el mismo modelo para `INTERACTION`, que es la tabla padre de `VIEW`, `REACTION`, `SHARE`, `REPORT`, y `MANAGEMENT`. También, y no es menor aclarar, se relaciona con `CONTENT` para poder vincular la interacción al contenido sobre el cual se hizo. Lamentablemente, por razones de scope, no está incluido en la app. Pero en el archivo `tp_integrador/docs/der.mmd` se pueden visualizar estas conexiones en formato mermaid.

Otro caso relevante es el del vector de embeddings. Éste no se asocia directo al contenido chunkeado sino a la tripla `UNIQUE (content_id, source_type, chunk_index)`, lo que permite que en un mismo contenido haya embeddings separados del título y el cuerpo para que la búsqueda pueda distinguir de dónde vino el resultado. De otra forma, quedaría una mezcla de ambos en el vector de resultados y no sería posible saber si el match vino del nombre o del texto del contenido

Eventos como filas

Las entidades que rastrean eventos, como `HISTORY` o `SEARCH`, almacenan un registro por evento con su timestamp en lugar de declarar un atributo que registre una cantidad dentro de `CONTENT`. Esto delega la responsabilidad en la entidad que realmente representa ese evento más que en el punto de entrada que es el contenido. A su vez, las métricas se pueden derivar con agregaciones en tiempo de consulta y no se duplican. Por ejemplo:

```
-- 3. Agregacion: actividad por contenido.
-- Pregunta: que contenidos concentran mas visualizaciones e interacciones?
SELECT
  c.id,
  c.title,
  COUNT(DISTINCT history.id) AS views,
  COUNT(DISTINCT interaction.id) AS interactions,
  COUNT(DISTINCT CASE WHEN interaction.interaction_type = 'reaction' THEN
interaction.id END) AS reactions,
  COUNT(DISTINCT CASE WHEN interaction.interaction_type = 'share' THEN
interaction.id END) AS shares
FROM CONTENT AS c
```



```
LEFT JOIN HISTORY AS history ON history.content_id = c.id
LEFT JOIN INTERACTION AS interaction ON interaction.content_id = c.id
GROUP BY c.id, c.title
ORDER BY views DESC, interactions DESC;

tp_integrador/sql/consultas.sql
```

Sin embargo, a gran escala, esto podría devenir en performance degradada porque `HISTORY` o `INTERACTION` podrían crecer demasiado. Un paliativo de esto sería crear vistas materializadas o precalcular métricas en alguna tabla dedicada

Desnormalizaciones deliberadas

Hubo algunos casos donde preferimos repetir información para agilizar consultas y tener mejor representación. Encontramos tres casos donde incurrimos en desnormalización pero nos pareció mejor mantenerlo:

1. `CONTENT.creator_id`: Si el usuario crea un contenido, queda almacenado como estado (en `CONTENT.creator_id`) y como evento (en `INTERACTION(user_id=U ...)`). Esto puede provocar que si se crea un contenido pero no se pasa por el flujo de creación esperado, éste no registre el evento y genere una inconsistencia. Vale decir, sin embargo, que la inconsistencia quedaría del lado de la aplicación y no de la base de datos. El ahorro de duplicar en `CONTENT` esta key tiene el valor de que hace más directo llegar a `USER` desde el contenido sin tener que hacer el flujo:

```
FROM CONTENT c
JOIN INTERACTION i ON i.content_id = c.id
                     AND i.interaction_type = management
JOIN MANAGEMENT m  ON m.interaction_id = i.id
                     AND m.action_type = create
JOIN usuario u      ON u.id = i.user_id
```

Además, como contraparte, hace que el mapeo entre creador y contenido sea inmune a la inconsistencia antes mencionada. También permitiría agregar un índice sobre ese atributo para el caso de uso típico de filtrar contenido por creador.

2. `CONTENT.content_type`: Duplica la información que se podría derivar de ver en qué tabla hija existe el `CONTENT.id`. Pero preservarla acá evita tener que hacer lo que hoy son 6 joins pero podrían ser N joins en el futuro. Para asegurar que esta relación es consistente, también se agregaron `CHECKS` en cada una de las tablas hijas en las que se asegura que no se pueda, por ejemplo, guardar un `'video'` en la tabla `POST` falseando su `content_type` sea vía app o vía inserción directa. Además, la app puede

utilizar ese atributo para renderizar desplegables o hacer operaciones que siempre van a ser consistentes.

3. `CONTENT_EMBEDDING.chunk_text`: El texto chunkeado que se guarda acá es una partición del texto de origen que existe en `POST.body`, `ARTICLE.full_text`, `COURSE.description` o `CONTENT.title`. Intencionalmente se está duplicando el texto para poder auditar el matcheo del proceso de búsqueda vectorial. Tiene como contraparte que cuando cambia el contenido hay que vectorizar de nuevo, pero es un costo asumido por la aplicación.

Fuera de estos tres casos relevados, entendemos que la base de datos está normalizada.

Selección y justificación de la tecnología utilizada

Tecnología	Rol	Justificación
PostgreSQL 16	Base relacional principal	Relaciones, integridad referencial, restricciones.
pgvector	Busqueda semántica	Embeddings dentro del mismo motor relacional, sin dependencia externa.
MinIO	Almacenamiento de objetos	API compatible con S3, desacopla binarios del motor relacional y cuenta con gran soporte en varias tecnologías.
FastAPI	Aplicacion web	Interfaz simple, tipado, documentacion automatica. Gran soporte y comunidad dentro del lenguaje Python.
all-MiniLM-L6-v2	Generacion de embeddings	Vectores de 384 dimensiones, buena relación calidad/velocidad.
Docker Compose	Orquestacion local	Tecnología de containerización que permite ejecución reproducible de PostgreSQL, pgAdmin y MinIO.

Se descartaron bases NoSQL y vectoriales independientes para mantener una única solución simple y consistente. Concentrar los datos en PostgreSQL simplifica la operación y conserva la consistencia entre datos relacionales y embeddings.

Implementación mínima

`tp_integrador/sql/ddl.sql` crea extensiones (`vector`, `uuid-oss`), tipos enumerados, tablas e índices. `tp_integrador/sql/seed.sql` inserta datos de prueba representativos en la base de datos relacional. Los datos de prueba binarios y de embeddings se cargan mediante scripts previos a la inicialización de la aplicación.

La aplicación FastAPI ([app/](#)) permite gestionar usuarios, categorías, etiquetas y contenidos mediante formularios web. Incluye endpoints para subir y visualizar fotos y videos, y un buscador semántico con que respeta la preferencia `result_list.max_results` como ejemplo de uso de preferencias del usuario para la aplicación (también se incluyó `theme light/dark` para preferencias, pero no se incluyó implementación de esto).

El proceso de embeddings ([embeddings/](#)) genera vectores y los guarda en `CONTENT_EMBEDDING` mediante upsert, procesando títulos, cuerpos de posts, textos completos de artículos, y descripciones de cursos, videos y fotos.

En un caso real, las fotos y videos son analizados por un modelo de ML que extrae descripción y transcripción para insertar en los campos `description` de las entidades `VIDEO` y `PHOTO` de forma tal de que estos tipos de contenido también puedan ser considerados en las búsquedas semánticas y las recomendaciones.

Los archivos binarios de ejemplo se cargan en MinIO mediante [setup.sh](#):

```
./setup.sh          # inicializa contenedores y sube archivos a MinIO
./setup.sh --purge  # elimina todos los datos incluidos los de MinIO
./run_app.sh        # levanta aplicación web
```

Datos de ejemplo y consultas representativas

Los datos de `tp_integrador/sql/seed.sql` permiten validar:

- usuarios con roles distintos y preferencias de interfaz y resultados;
- contenidos de todos los tipos con categorías y etiquetas;
- historial de visualizaciones, búsquedas y resultados de búsqueda;
- interacciones de distintos tipos (visualización, reacción, compartido, denuncia, gestion);
- embeddings generales a partir de textos y descripciones incluidos en el seed.

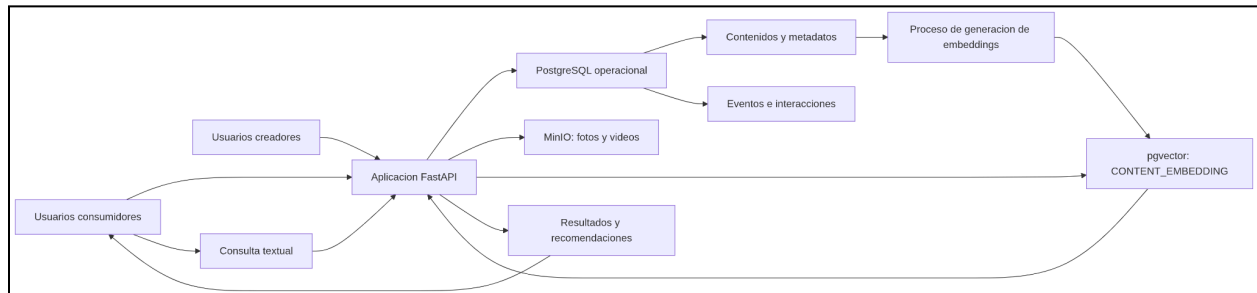
Los archivos binarios de ejemplo (`tp_integrador/data/`) son una imagen JPG y un video MP4 de ejemplo identificados por UUID, subidos al bucket assets de MinIO durante el setup.

`tp_integrador/sql/consultas.sql` contiene:

1. filtrado por categoría y tipo de contenido;
2. relación entre contenido, categoría y etiquetas;
3. agregación de visualizaciones e interacciones;
4. seleccion de contenidos segun preferencias e historial del usuario;
5. consulta con `EXPLAIN` para revisar el uso de índices;
6. búsqueda semántica con pgvector (similitud coseno sobre `CONTENT_EMBEDDING`).

Arquitectura general de datos

La solución utiliza una arquitectura de capas. PostgreSQL es el almacenamiento principal; pgvector resuelve la búsqueda semántica dentro del mismo motor; MinIO almacena los binarios de forma separada. No se agrega un Data Warehouse, Data Lake ni base vectorial independiente.



Arquitectura de datos. Puede encontrarse en formato PNG en tp_integrador/docs/arquitectura_datos.png

Capas

- **Fuentes y usuarios:** consumidores generan búsquedas, visualizaciones y reacciones; creadores ingresan y modifican contenidos.
- **Ingestión y aplicación:** FastAPI recibe formularios y consultas, valida datos y ejecuta operaciones sobre PostgreSQL y MinIO.
- **Almacenamiento operacional (PostgreSQL):** usuarios, roles, contenidos, categorías, etiquetas, preferencias, historial, búsquedas, interacciones y acciones de gestión.
- **Almacenamiento de objetos (MinIO):** archivos binarios; la base relacional guarda solo la URL de referencia.
- **Preparación para IA:** proceso Python que lee textos, genera embeddings y los guarda en `CONTENT_EMBEDDING`.
- **Consulta y consumo:** consultas SQL convencionales y vectoriales para recuperar y recomendar contenidos.

Una arquitectura simple es suficiente porque los casos de uso son reducidos, las relaciones entre entidades son importantes, se necesita consistencia entre contenidos e interacciones.

Seguridad, permisos y aislamiento

Como se adelantó en la sección [Decisiones de normalización en las relaciones](#), el modelo de roles que elegimos fue RBAC. Lo utilizamos como base para implementar el Row Level Security (RLS) con los roles definidos. La base del aislamiento que vamos a seguir es bloquear verticalmente. Es decir, que los roles garanticen ciertas operaciones sobre las tablas

más que acceso a ciertas tablas, ya que por diseño, cada usuario debe poder consultar todas las tablas para poder crear y leer contenido.

Creamos 4 tipos de usuario: Admin, Moderator, Editor, User y creamos las entradas correspondientes para cada uno

Usuario	Rol	history	search	report	users	content
alice_smith	User	2	1	0	1	6
bob_jones	User	1	1	0	1	6
charlie_brown	Moderator	1	0	1	5	6
diana_prince	Editor	0	0	0	1	6
admin_user	Admin	4	2	1	5	6

Nótese que el usuario `admin` que se usa para levantar el docker y navegar en la app está excluido de todo esto. Es una decisión deliberada para oficiar de *sandbox* y poder hacer todas las operaciones posibles. En la base de datos, el `admin_user` representa al admin real, que sí está sujeto al RLS

Por otro lado, en postgres creamos los roles y permisos que se asocian con los roles de negocio antedichos:

Own — únicamente sus propias filas All — todas las filas — sin acceso

MATRIZ DE PERMISOS EFECTIVOS

Recurso	Operación	User	Editor	Moderator	Admin	 (app)
CONTENT	leer	All	All	All	All	All
	crear	Own	Own	Own	All	All
	editar	Own	All	All	All	All
	eliminar	Own	All	All	All	All
	reasignar autoría	—	—	—	All	All
Subtipos video, photo, article, post, course, document	leer	All	All	All	All	All
	escribir	Own	All	All	All	All
CONTENT_TAG	leer	All	All	All	All	All
	escribir	Own	All	All	All	All
HISTORY · SEARCH PREFERENCE USER_PREFERENCE	leer	Own	Own	Own	All	All
	escribir	Own	Own	Own	Own	All
SEARCH_RESULT	leer	Own	Own	Own	All	All
INTERACTION + view, reaction, share, management	leer	Own	Own	All	All	All
	escribir	Own	Own	Own	Own	All
REPORT	leer	Own	Own	All	All	All
"user"	leer	Own	Own	All	All	All
	password_hash	—	—	—	—	All
	escribir	—	—	—	—	All
USER_ROLE	leer	Own	Own	All	All	All
	escribir	—	—	—	All	All
ROLE · CATEGORY · TAG	leer	All	All	All	All	All
	escribir	—	—	—	—	All
CONTENT_EMBEDDING	leer	All	All	All	All	All
	escribir	—	—	—	—	All

Matriz de permisos para roles y usuarios. Elaboración propia

Por el scope del trabajo, los roles de postgres se crearon para `app_user` como su propio username. Esta implementación tiene límites. Particularmente, impide compartir un pool de conexiones, convierte el alta de usuario en una operación DDL y utiliza el catálogo global del cluster como directorio de usuarios. Ante un volumen mayor, debería hacerse una migración que consistiría en adoptar un rol de aplicación único y fijar la identidad por transacción con `set_config('app.user_id', ..., true)`. Las políticas no requerirían modificación, ya

que están expresadas como una función de contexto y no están *hardcodeadas* contra los nombres de los roles.

Las pruebas de que estas operaciones se restringen y permiten de manera satisfactoria se encuentran en el archivo `tp_integrador/rls_check.sh` que se puede ejecutar vía `bash`.

Escalabilidad y rendimiento

Tablas de mayor crecimiento esperado: `HISTORY`, `INTERACTION`, `SEARCH`, `SEARCH_RESULT` y `CONTENT_EMBEDDING`. Los datos de catálogo (categorías, etiquetas, roles) crecerían mucho menos.

Índices incluidos: sobre claves foráneas y columnas frecuentemente consultadas, mas un índice HNSW sobre `CONTENT_EMBEDDING.embedding` para similitud coseno. Los índices deben revisarse con `EXPLAIN` y métricas reales.

Consultas más sensibles al crecimiento: búsqueda semántica, historial de usuario por fecha, conteo de interacciones por contenido, contenidos recomendados por categoría y recuperación de contenidos más recientes.

Estrategias de escalabilidad posibles:

- Particionar `HISTORY`, `INTERACTION` y `SEARCH` por fecha.
- Precalcular métricas de popularidad y crear vistas materializadas.
- Utilizar caché para contenidos y recomendaciones repetidas.
- Procesar embeddings de manera asincrónica.
- Separar cargas analíticas de las operaciones transaccionales.
- Agregar réplicas de lectura.
- Separar pgvector en un servicio vectorial independiente si fuera necesario.
- Reemplazar el proxy FastAPI por un CDN o bucket público para servir binarios a escala.

Conclusiones

PostgreSQL con pgvector permite resolver el caso con una arquitectura compacta y consistente. MinIO desacopla el almacenamiento de binarios del motor relacional, siguiendo el patrón habitual de separar datos estructurados y archivos no estructurados.

El modelo distingue datos operacionales, datos semiestructurados (JSONB), datos no estructurados (binarios en MinIO) y representaciones vectoriales (embeddings en pgvector). Esta separación facilita la comprensión, el mantenimiento y la evolución futura del sistema.

Las limitaciones actuales son propias del alcance académico: sin recomendador avanzado, autenticación, permisos efectivos, moderación automática ni almacenamiento analítico separado. Como mejoras futuras se podrían agregar hashing adaptativo de contraseñas, autorización basada en roles de PostgreSQL o RLS, métricas precalculadas, procesamiento asincrónico de embeddings y particionamiento de tablas de eventos.